

说明

本文档面向使用 CH32V467 系列微控制器的嵌入式开发人员，详细说明如何通过修改链接脚本（Link.ld）和利用启动文件（startup）的钩子函数，将指定模块（例如 CoreMark 性能测试代码）从内部 Flash 搬运到 PSRAM 中运行，从而利用 PSRAM 的大容量和高带宽优势提升代码执行效率。本应用笔记结合 EVT 中的 EVT\EXAM\APPLICATION\RunInPSRAM 例程进行说明，并基于实际工程文件（包括 Link.ld、startup_ch32v4x7.s 和 PSRAM_Prepare 实现）给出详细指导。

适用范围

适用范围	系列
通用 MCU	CH32V467

目录

说明	i
目录	ii
第 1 章 内存布局与链接脚本解析	3
1.1 内存区域定义	3
1.2 关键段 (Section) 定义	3
1.3 数据段与 BSSc	4
1.4 栈与堆	4
第 2 章 启动流程与 PSRAM 初始化	5
2.1 原始启动流程回顾	5
2.2 PSRAM_Prepare 函数详解	5
2.3 代码搬运的 C 语言实现	6
第 3 章 编译与烧录配置	7
3.1 编译设置	7
3.2 烧录方法	7
3.3 运行时序	7
第 4 章 问题与解决方案	8
4.1 常见问题排查	8
4.2 注意事项	8
历史版本	9
声明	10

第1章 内存布局与链接脚本解析

1.1 内存区域定义

在链接脚本（.ld）中，通过 MEMORY 命令定义了三个独立的内存区域：

```
MEMORY
{
    FLASH (rx) : ORIGIN = 0x00000000, LENGTH = 576K
    RAM (xrw)  : ORIGIN = 0x20000000+1024, LENGTH = 136K-1K
    PSRAM (xrw): ORIGIN = 0x80000000, LENGTH = 1M
}
```

FLASH：起始地址 0x00000000，容量 576KB（对应 CH32V467 最大 Flash 配置）。属性 rx（可读、可执行），存放默认代码和常量。

RAM：起始地址 0x20000000+1024（即偏移 1KB），容量 135KB（136K-1K）。保留的 1KB 可用于 Bootloader 或特殊系统预留。属性 xrw（可读、写、执行），存放全局变量和堆栈。

PSRAM：起始地址 0x80000000，容量 1MB（可根据实际芯片容量调整）。属性 xrw，表明代码可在其中运行且数据可读写。

1.2 关键段（Section）定义

链接脚本中专门为 PSRAM 设计了以下几个段，实现代码的加载地址（LMA）与运行地址（VMA）分离：

```
R /* ① 定义 PSRAM 代码段的运行起始地址（VMA） */
.dalign :
{
    . = ALIGN(4);
    PROVIDE(_psram_vma = .);
} >PSRAM AT>FLASH

/* ② 定义 PSRAM 代码段的加载起始地址（LMA） */
.dlalign :
{
    . = ALIGN(4);
    PROVIDE(_psram_lma = .);
} >FLASH AT>FLASH

/* ③ PSRAM 代码段本体 */
.psrtext :
{
    . = ALIGN(4);
    ./User/coremark/(*.text)
    ./User/coremark/(*.text.*)
    ./User/coremark/(*.rodata)
```

```

./User/coremark/*(.rodata*)
./User/coremark/*(.gnu.linkonce.t.*)
. = ALIGN(4);
} >PSRAM AT>FLASH

/* ④ 标记加载结束地址（LMA 结束） */
.dlalign :
{
    . = ALIGN(4);
    PROVIDE(_psram_lma_end = .);
} >FLASH AT>FLASH AM (xrw) : ORIGIN = 0x20000000+1024, LENGTH = 136K-1K
    PSRAM (xrw): ORIGIN = 0x80000000, LENGTH = 1M
}

```

- `_psram_vma`: `.psramtext` 在 PSRAM 中运行时的起始地址（最终为 `0x80000000` 对齐后的值）。
- `_psram_lma` 和 `_psram_lma_end`: `.psramtext` 在 Flash 中存放的起始和结束地址，用于启动时复制。
- `.psramtext` 段收集了 `./User/coremark/` 目录下所有目标文件的 `.text`、`.text.*`、`.rodata`、`.rodata*` 以及 `.gnu.linkonce.t.*` 节。这些内容同时存在于 Flash（备份）和 PSRAM（运行）。

1.3 数据段与 BSS

```

.data : { ... } >RAM AT>FLASH
.bss : { ... } >RAM AT>FLASH

```

所有可读写全局变量、静态变量均放置于内部 RAM 中，其初始值存储在 Flash 中，由启动代码在复位后复制。

`.bss` 段（未初始化全局变量）在 RAM 中分配，启动代码负责清零。

1.4 栈与堆

```

.stack ORIGIN(RAM) + LENGTH(RAM) - __stack_size :
{
    PROVIDE( _heap_end = . );
    . = ALIGN(4);
    PROVIDE( _susrstack = . );
    . = . + __stack_size;
    PROVIDE( _eusrstack = . );
} >RAM

```

- 栈顶位于 RAM 末尾 (`ORIGIN(RAM) + LENGTH(RAM)`)，向下生长。
- 栈大小固定为 2048 字节 (`__stack_size = 2048`)。
- 符号 `_eusrstack` 作为栈顶地址，在启动文件中用作 `sp` 初始值

第2章 启动流程与 PSRAM 初始化

启动文件 startup_ch32v4x7.s 是芯片复位后执行的第一个代码，负责硬件初始化、数据搬运和跳转到 main。为支持 PSRAM 运行，我们在 C 层实现了一个 PSRAM_Prepare 函数，由启动文件在复位流程中调用，该函数完成 PSRAM 硬件初始化和代码搬运。

2.1 原始启动流程回顾

原始 handle_reset 函数（位于 .text.handle_reset 段）执行以下步骤：

1. 设置全局指针 gp。
2. 设置栈指针 sp 为 _eusrstack。
3. 复制 .data 段从 Flash 到 RAM。
4. 清零 .bss 段。
5. 配置系统寄存器（Prefetch、中断嵌套、mstatus、mtvec 等）。
6. 调用 PSRAM_Prepare 初始化 PSRAM 硬件（该函数由用户实现，必须位于 Flash 中）。
7. 跳转到 main（通过 mret 从机器模式切换到用户模式）。

2.2 PSRAM_Prepare 函数详解

PSRAM_Prepare 是启动流程中调用的核心函数，其典型实现如下（位于 PSRAM.c）：

```
void PSRAM_Prepare() {
    SystemCoreClockUpdate();           // 获取当前系统时钟
    PSRAM_200MHz_HSE();                 // 配置 PLL 为 200MHz（使用 HSE）
    SystemCoreClockUpdate();           // 更新时钟变量
    Delay_Init();                       // 初始化延时函数
    PSRAM_INIT();                      // 初始化 PSRAM 控制器（时序、模式等）
    USART_Printf_Init(115200);          // 初始化串口（用于调试输出）

    // 利用链接脚本导出的符号进行代码搬运
    uint32_t *rp = _psram_lma, *wp = _psram_vma;
    printf("%#x %#x %#x\n", rp, wp, _psram_lma_end);
    Delay_Ms(10);
    while (rp < _psram_lma_end) {
        *wp = *rp;
        wp++;
        rp++;
    }
}
```

各步骤说明：

- 时钟配置：PSRAM_200MHz_HSE() 将系统时钟设置为 200MHz（HSE 25MHz * 8），以满足 PSRAM 的高速访问需求。
- PSRAM 硬件初始化：PSRAM_INIT() 配置时序参数（trc, tcph, txlpd）、容量（PSRAM_CAPCFG_32M）等，并使能 PSRAM 外设时钟。
- 代码搬运：使用三个外部符号（_psram_lma, _psram_vma, _psram_lma_end），通过 C 语言循环将 .psramtext 从 Flash 复制到 PSRAM。这是关键步骤，完成运行时代码加载。
- 调试输出：打印三个符号地址，便于验证。

2.3 代码搬运的 C 语言实现

搬运循环采用字（4 字节）复制，简单高效：

```
uint32_t *rp = _psram_lma;    // 源地址 (Flash)
uint32_t *wp = _psram_vma;    // 目标地址 (PSRAM)
while (rp < _psram_lma_end) {
    *wp = *rp;
    wp++;
    rp++;
}
```

优点：

- 无需修改汇编启动文件，所有逻辑在 C 层实现，便于维护和调试。
- 复制时机在 PSRAM 硬件初始化之后，确保写入安全。
- 使用指针操作，可利用编译器优化提高复制速度。

第3章 编译与烧录配置

3.1 编译设置

- 确保工程中包含了 User/coremark 下的所有源文件，以及 PSRAM.c、PSRAM_Init.c 等。
- 链接脚本使用 Ld/Linkld（已按上述内容配置）。
- 编译后生成 .elf、.hex 或 .bin 文件。

3.2 烧录方法

由于代码分布在 Flash 和 PSRAM 两个区域，烧录时需注意：

- Flash（0x00000000）：必须烧录完整的固件镜像，其中包含 .psramtext 的原始备份。
- PSRAM（0x80000000）：无需预先烧录，因为启动时 PSRAM_Prepare 会从 Flash 复制过来。

3.3 运行时序

- 芯片上电，从 Flash 的 0x00000000 执行 .init 段，进入 _start → handle_reset。
- 完成常规初始化（栈、数据、BSS、系统寄存器）。
- 调用 PSRAM_Prepare（Flash 中执行）：
 - ◆ 提升系统时钟至 200MHz。
 - ◆ 初始化 PSRAM 硬件。
 - ◆ 将 .psramtext 从 Flash 复制到 PSRAM。
- 返回 handle_reset，通过 mret 跳转至 main（此时 main 位于 coremark/main.c，已在 PSRAM 中，正常执行）。

第4章 问题与解决方案

4.1 常见问题排查

问题现象	可能原因	解决方法
上电后程序跑飞，PC 指向 0x80000000 但内容全零	PSRAM 未初始化或复制未执行	检查 PSRAM_Prepare 是否被调用；确认 PSRAM_INIT 执行成功（可读取状态寄存器）。
搬运循环中写入 PSRAM 导致硬故障	PSRAM 控制器未使能或时序错误	完善 PSRAM_INIT 的时序参数，参考芯片手册；确认 PSRAM 基地址为 0x80000000。
_psram_lma 与 _psram_lma_end 相等	coremark 目录下源文件未被编译	检查工程设置，确保 coremark 文件参与编译。
PSRAM_Prepare 自身无法执行（PC 异常）	该函数被误放在 coremark 目录，链接到 PSRAM	将 PSRAM.c 等文件移出 coremark 目录，或使用 __attribute__((section(".text"))) 强制放入 Flash。
复制后代码仍从 Flash 执行（PC 不变）	main 不在 coremark 目录，或链接未将 main 放入 PSRAM	确认 main 位于 coremark/main.c；若不在，可将 main 函数加上 __attribute__((section(".psramtext")))。
PSRAM 中运行速度未提升	PSRAM 初始化未完成或时钟频率不足	检查 PSRAM 缓存控制寄存器，使能缓存；确认系统时钟为 200MHz。

4.2 注意事项

- PSRAM_Prepare 及其所有依赖函数必须留在 Flash 中
这些函数所在的源文件（如 PSRAM.c、PSRAM_Init.c）绝对不能放在 User/coremark 目录下，否则它们会被链接到 .psramtext 段，导致在 PSRAM 未就绪时执行，引发硬故障。
- PSRAM 容量配置必须与实际芯片匹配
PSRAM_INIT 中的 PSRAM_CAPCFG_32M 等宏定义需根据芯片实际 PSRAM 容量选择（例如 4MB、8MB）。错误配置可能导致访问异常或只能使用部分空间。
- 中断与调试输出
USART_Printf_Init 和 printf 的实现若依赖于某些外设，请确保这些外设的时钟已在之前使能，且其代码不在 PSRAM 段中。通常串口驱动位于标准外设库，不会受影响。
- 复制循环的时间开销
如果 .psramtext 较大（例如数十 KB），复制过程会占用一定启动时间，但通常可接受。若需加速复制，可考虑使用 DMA，但需注意 DMA 配置复杂度。

历史版本

更新内容

日期	版本	变更内容
2026/08/25	V1.0	初版发行

声明

本手册版权所有为南京沁恒微电子股份有限公司（Copyright © Nanjing Qinhang Microelectronics Co., Ltd. All Rights Reserved），未经南京沁恒微电子股份有限公司书面许可，任何人不得因任何目的、以任何形式（包括但不限于全部或部分地向任何人复制、泄露或散布）不当使用本产品手册中的任何信息。

任何未经允许擅自更改本产品手册中的内容与南京沁恒微电子股份有限公司无关。

南京沁恒微电子股份有限公司所提供的说明文档只作为相关产品的使用参考，不包含任何对特殊使用目的的担保。南京沁恒微电子股份有限公司保留更改和升级本产品手册以及手册中涉及的产品或软件的权利。

参考手册中可能包含少量由于疏忽造成的错误。已发现的会定期勘误，并在再版中更新和避免出现此类错误